

MODULE 6

Dataverse Fundamentals

for Vibe Apps

Enterprise Training | Microsoft Power Platform | 4-Hour Session

Ruthran Raghavan

Chief AI Scientist

www.ruthranraghavan.com

Tables & Columns

Relationships

CRUD Operations

Vibe Apps + AI

Best Practices

About Your Instructor



Ruthran Raghavan

Chief AI Scientist

www.ruthranraghavan.com

- Microsoft Power Platform & Dataverse Architecture
- AI & Copilot Integration in Business Applications
- Enterprise Low-Code / Vibe Coding Solutions
- Cloud Data Strategy & Governance
- Digital Transformation & Training Leadership

Course Agenda

01

Introduction to Dataverse

30 min

02

Tables, Columns & Data Types

40 min

03

Relationships & Lookups

35 min

04

CRUD Operations & Power Fx

35 min

05

Schema Design & Best Practices

30 min

06

Dataverse + Vibe Apps (AI)

30 min

07

Knowledge Check & Capstone Lab

40 min

Learning Objectives

By the end of this module, you will be able to:

 Explain what Microsoft Dataverse is and how it compares to traditional databases

 Create and configure Tables, Columns, and Rows in Dataverse

 Design and implement Relationships (1:1, 1:N, N:N) between tables

 Perform CRUD operations using Power Apps and Power Fx

 Apply Dataverse security roles, business rules, and governance

 Integrate Dataverse with Vibe Apps and Microsoft Copilot

 Build a complete HR Management System from scratch

01

Introduction to Dataverse

Understanding the foundation of the Microsoft Power Platform data layer

Why Data Matters in Modern Business

2.5

Quintillion

Bytes of data created daily worldwide

80%

of business

decisions are now data-driven

73%

of organizations

report data silos as a top challenge

3.5x

more likely

data-driven companies outperform peers

The Excel Problem — When Spreadsheets Become a Liability



Version Chaos

Multiple copies of the same file:
'employee_list_v3_FINAL_REAL.xlsx'



No Concurrent Access

Only one person can edit at a time —
collaboration is blocked



No Security Model

Anyone with file access can see all data,
including salaries



No Audit Trail

Impossible to know who changed what
and when



No Business Logic

No validation rules — wrong data types,
empty required fields



Scalability Limits

Excel slows to a crawl beyond 100k rows;
crashes are common

Choosing the Right Data Platform

Feature	Excel	SharePoint Lists	SQL Database	Dataverse
Concurrent users	✗ 1 at a time	✓ Multiple	✓ Many	✓ Unlimited
Security / RBAC	✗ File-level only	⚠ Basic	⚠ Manual config	✓ Built-in roles
Business rules	✗ None	✗ None	⚠ Code required	✓ No-code rules
Power Apps native	⚠ Limited	⚠ Limited	⚠ Custom connector	✓ Native
Relationships	✗ Manual	⚠ Lookups only	✓ Full FK support	✓ Native model
Copilot / AI	✗	✗	✗	✓ Native Copilot
Audit logging	✗	⚠ Version history	⚠ Manual	✓ Built-in
Scalability	⚠ ~100k rows	⚠ 30M items	✓ Millions	✓ Enterprise scale

What is Microsoft Dataverse?

Microsoft Dataverse is a secure, cloud-based, low-code data platform that allows you to store and manage data used by business applications. It provides enterprise-grade security, rich metadata, built-in business logic, and native integration with the entire Microsoft Power Platform.



Secure

Role-based access control,
encryption, compliance



Cloud-Native

Managed by Microsoft Azure,
always available



Low-Code

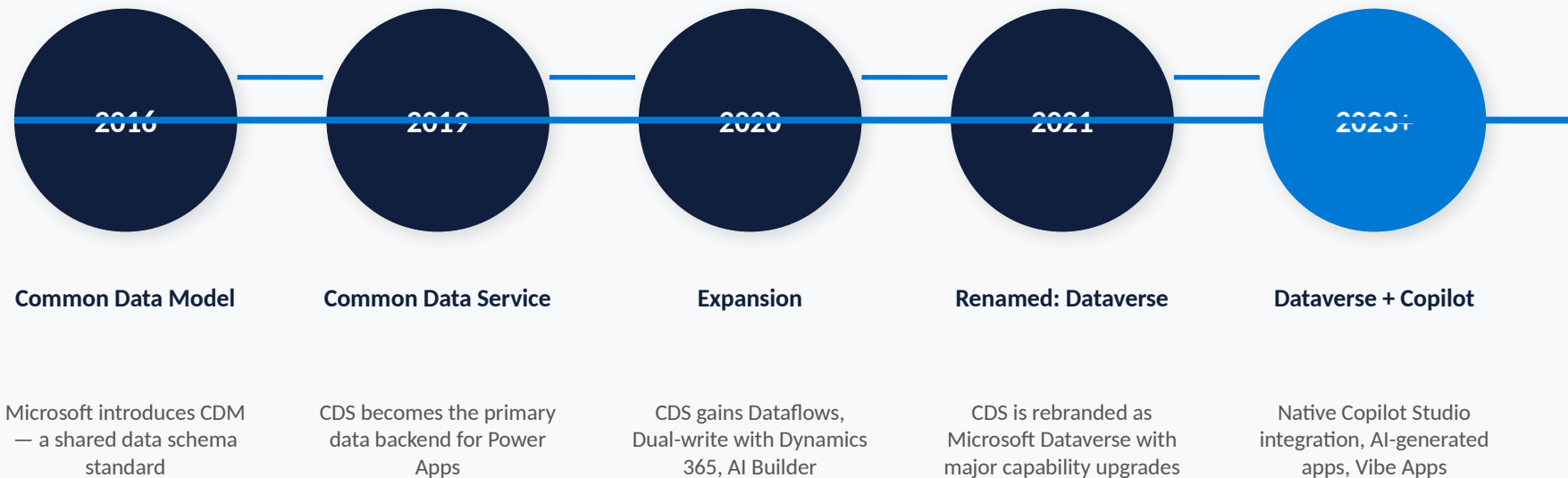
No SQL needed — visual table
designer



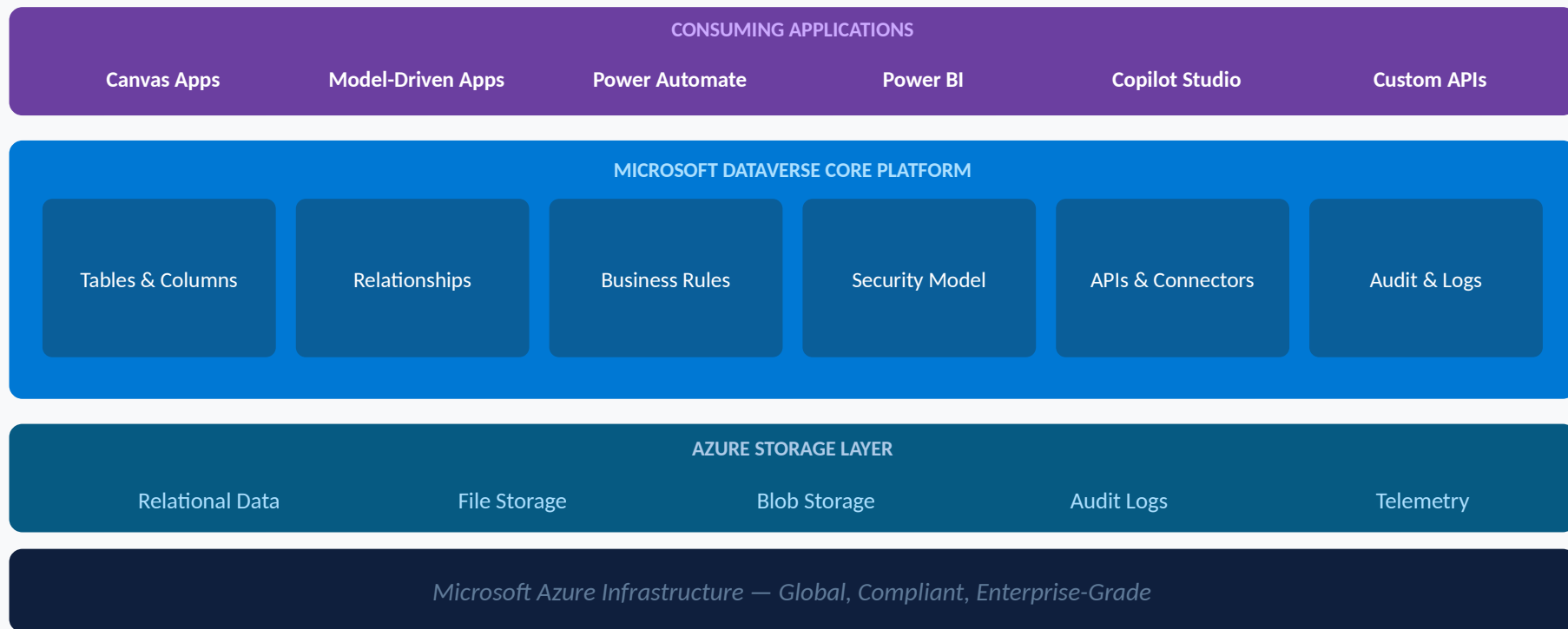
Integrated

Native to Power Apps, Power
Automate, Copilot

Evolution: From Common Data Service to Dataverse



Dataverse Architecture Overview



Dataverse Inside Microsoft Form



Dynamics 365



Power Apps



Power Automate



Power BI



Copilot Studio

Microsoft
Dataverse

02

Tables, Columns & Data Types

The building blocks of your Dataverse data model

Tables — The Foundation of Dataverse

A Table in Dataverse is equivalent to a table in a relational database — it stores structured data about a specific entity (person, object, event, or concept). Unlike a database table, a Dataverse table comes with built-in metadata, security, and APIs out of the box.

Employee Table (Example)

Employee ID	Full Name	Department	Job Title	Email	Start Date
EMP-001	Priya Sharma	Engineering	Senior Dev	priya@co.com	01/15/2022
EMP-002	Rajesh Kumar	Finance	Analyst	rajesh@co.com	03/20/2023
EMP-003	Anita Singh	HR	HR Manager	anita@co.com	07/01/2021

 Each row = one Record | Each column = one Field / Column | Each table = one Entity

Standard Tables vs Custom Tables

Standard Tables (Out-of-the-Box)

- Pre-built by Microsoft for common scenarios
- Contact, Account, Lead, Opportunity (Dynamics 365)
- User, Team, BusinessUnit (System tables)
- Cannot be deleted; can be extended with new columns
- Follow Microsoft's Common Data Model schema

Custom Tables (You Build These)

- Created by you for your specific business needs
- Examples: Employee, Department, Project, Training
- Fully customizable: columns, relationships, views
- Can be made available offline (Offline-capable)
- Best practice: prefix with your org abbreviation (e.g., cr7a_Employee)

 *Tip: Always check if a Standard Table meets your needs before creating a Custom Table*

Columns — Defining Your Data Structure



Text

Employee Name, Email Address



Multiline Text

Job Description, Notes



Number (Whole)

Age, Quantity, Employee Count



Decimal Number

Rating (4.5), GPA (3.75)



Currency

Salary, Budget, Revenue



Date Only

Start Date, Birth Date



Date & Time

Created On, Meeting DateTime



Choice

Status, Department, Priority



Lookup

Department (→ Dept Table)



Boolean (Yes/No)

Is Active, Remote Worker



File

Resume, Contract PDF



Image

Employee Photo, Product Image

Choice Columns — Dropdown Lists Done Right

Choice columns let you define a fixed set of allowable values for a column. Users select from the list — no free-text errors.

SINGLE CHOICE

Example: Employment Status

Active

On Leave

Resigned

Terminated

Probation

MULTI-SELECT CHOICE

Example: Skills (select all that apply)

✓ Power Apps

✓ Power Automate

Power BI

✓ Dataverse

SQL

03

Relationships & Lookups

Connecting your tables to model real-world business entities

Why Relationships Matter

Relationships allow you to link data across multiple tables — eliminating duplication, ensuring consistency, and enabling rich queries across entities.

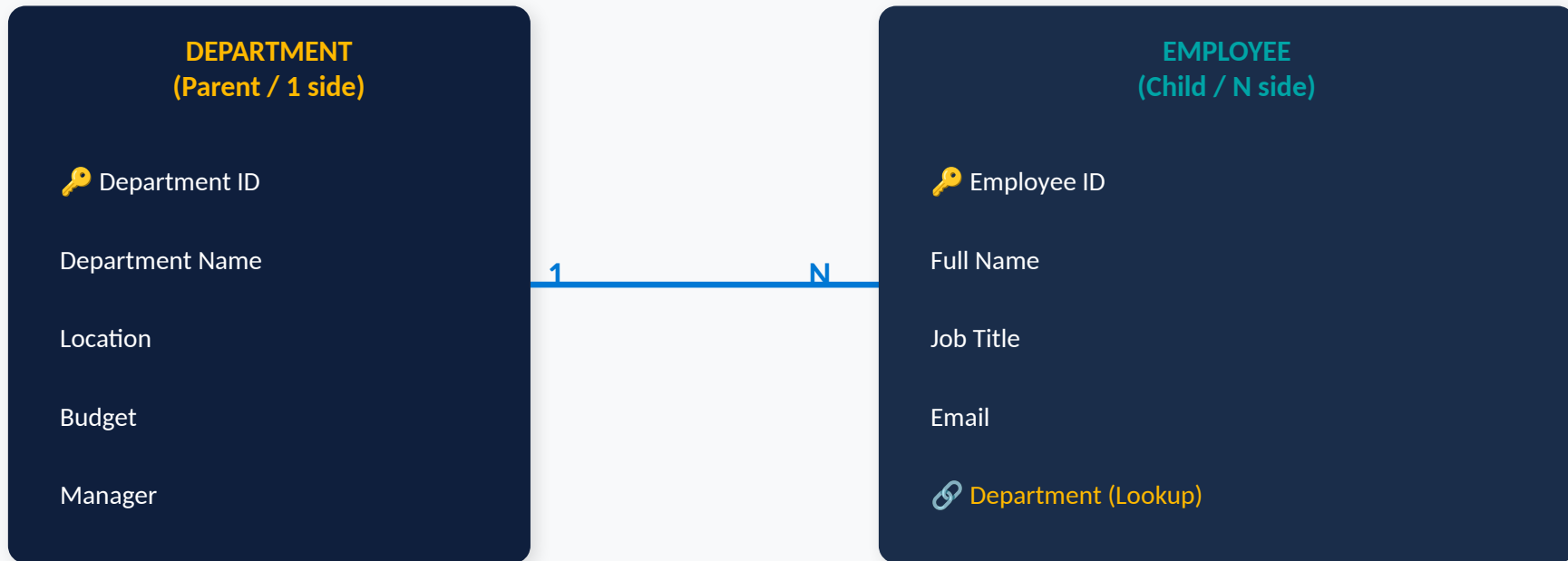
✗ WITHOUT Relationships

- Department name typed manually in each employee record
- Typos: 'Enginerig', 'Eng', 'Engineering Dept'
- Department phone changes → update 200 employee records
- No way to find all employees in a department reliably

✓ WITH Relationships

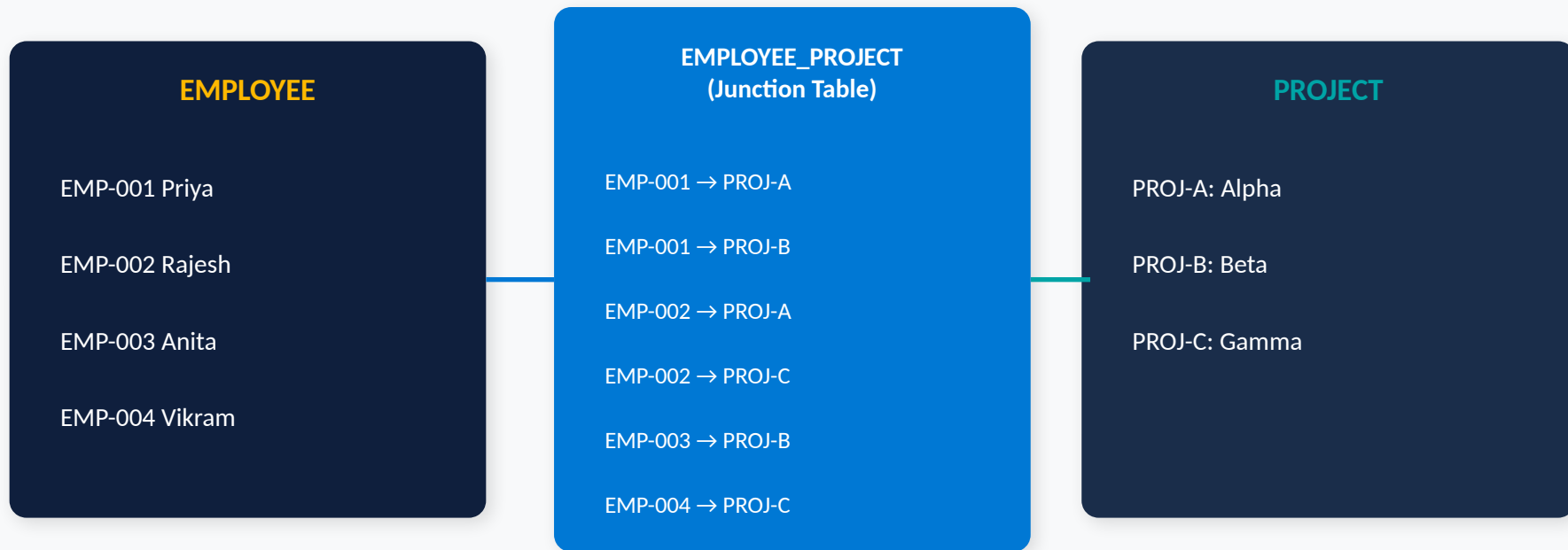
- Employee table Lookup → Department table
- Department name comes from one authoritative source
- Update department once — all linked employees reflect the change
- Easily filter all employees in Engineering with one click

1:N Relationship — One-to-Many (Most Common)



One Department → Many Employees | One Employee → One Department

N:N Relationship — Many-to-Many



One Employee → Many Projects | One Project → Many Employees
Dataverse creates a hidden intersection table automatically for N:N relationships.

04

CRUD Operations & Power Fx

Creating, Reading, Updating and Deleting records with Power Apps

CRUD Operations in Dataverse


CREATE

Add a new record to a Dataverse table

Power Fx: `Patch(Employees, Defaults(Employees), {FullName: txtName.Text, Department: ddDept.Selected})`


READ

Retrieve records from a table (with filtering)

Power Fx: `ClearCollect(colEmployees, Filter(Employees, Status = 'Active'))`


UPDATE

Modify an existing record

Power Fx: `Patch(Employees, LookUp(Employees, EmployeeID = lblID.Text), {JobTitle: txtTitle.Text})`


DELETE

Remove a record from the table

Power Fx: `Remove(Employees, LookUp(Employees, EmployeeID = galEmployees.Selected.EmployeeID))`

Power Fx Deep Dive — Creating & Updating Records

Patch() — The Swiss Army Knife of Power Fx

✓ CREATE a New Employee Record

```
Patch(Employees, Defaults(Employees),  
  {FullName:  
    txtFullName.Text, Department:  
    ddDepartment.Selected, StartDate:  
    dpStartDate.SelectedDate})
```

← Defaults() creates a blank row

← Field assignments

← Date picker value

UPDATE an Existing Record (change job title)

```
Patch(Employees, galEmployees.Selected, {JobTitle: txtNewTitle.Text, LastModified: Now()})
```

galEmployees.Selected references the currently selected gallery item — no LookUp needed.

Reading Data — Filter, LookUp, Search

Filter() — Get Multiple Records

```
Filter(Employees, Department.DepartmentName = "Engineering", Status = 'Active')
```

💡 Returns a table of all active Engineering employees. Use in Gallery.Items.

LookUp() — Get a Single Record

```
LookUp(Employees, EmployeeID = "EMP-001")
```

💡 Returns one record. Use when you know the ID of the record you need.

Search() — Keyword Search

```
Search(Employees, txtSearch.Text, "FullName", "Email")
```

💡 Searches text fields for the keyword. Perfect for a search bar in your app.

CountRows() — Count Records

```
CountRows(Filter(Employees, Status = 'Active'))
```

💡 Returns the number of active employees. Use in a label to show totals.

05

Security, Schema & Best Practices

Protecting your data and designing for scale

Dataverse Security Model



Environment Security

Who has access to the Dataverse environment itself?



Security Roles

What tables and columns can a user Create / Read / Update / Delete?



Field-Level Security

Can specific users see the Salary column? (Column permissions)



Record-Level Sharing

Share specific records with individuals or teams

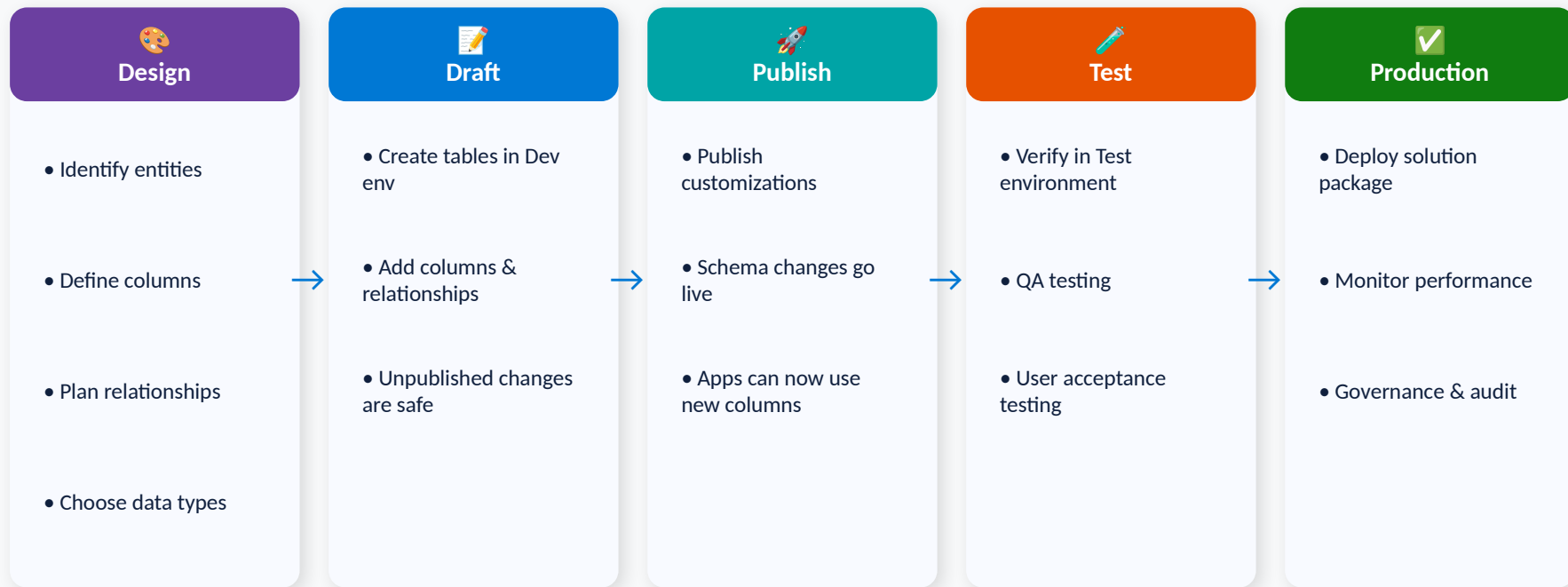


Business Unit Hierarchy

Organizational hierarchy — users only see their own BU data

Security is layered — each level adds finer-grained control. All layers work together.

Schema Lifecycle — From Design to Production



Best Practice: NEVER make schema changes directly in Production. Always use: Development → Test → Production.

Best Practices — Naming Conventions

Consistent naming prevents confusion and makes your data model maintainable at scale.

Element	✓ Good Example	✗ Bad Example	Rule
Tables (Custom)	cr7a_Employee	emp, MY_EMP, tbl_employees	Publisher prefix + PascalCase entity name
Columns (Custom)	cr7a_EmployeeID	EmpID, emp_id, ID	Publisher prefix + PascalCase attribute
Global Choices	cr7a_EmployeeStatus	Status, EmpStatus	Publisher prefix + entity + attribute
Relationships	cr7a_Employee_Department	rel1, Emp_Dept	Parent table _ Child table
Solutions	HRSysystem_v1_0_0	Solution1, MyApp	Business domain + version number

 The publisher prefix (e.g., cr7a_) is assigned automatically when you create a solution.

Business Rules — No-Code Data Validation

Business Rules let you enforce data logic without writing a single line of code. They run on the server side — protecting your data regardless of which application is used.

Set Field Value

Table/
Form

If Status = 'Terminated', set End Date = Today()

Set Column Requirement

Table/
Form

If Status = 'Active', Email is Required

Show/Hide Column

Form
only

If Employee Type / 'Contractor', hide Contract End Date

Validate Data

Table

If Salary < 0, show error 'Salary cannot be negative'

Lock/Unlock Column

Form
only

If record is Approved, lock all financial fields

Set Column Visibility

Form
only

Show Bonus column only for Management employees

06

Dataverse + Vibe Apps

AI-powered application generation with Microsoft Copilot

What Are Vibe Apps?

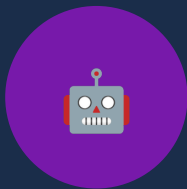
Vibe Apps (also called Vibe Coding) is the practice of using AI — specifically Microsoft Copilot in Power Apps — to describe your app in plain language and let AI generate the complete application, including Dataverse tables, relationships, forms, galleries, and navigation.



STEP 1

Describe Your App

Type: 'I need an HR app to track employees, departments, and leave requests'



STEP 2

Copilot Designs Schema

AI suggests Dataverse tables, columns, and relationships



STEP 3

Review & Approve

You review the schema, add/remove columns, adjust data types



STEP 4

App is Generated

Copilot creates the full Canvas App with forms, galleries, navigation



STEP 5

Customize & Deploy

You refine the UI, add business rules, and publish to your organization

Vibe Apps + Dataverse: The AI-Powered Stack

User (Natural Language Prompt)

"Build me an employee leave management app with approval workflow"

Microsoft Copilot (AI Engine)

Interprets intent → Designs schema → Generates app structure

Power Apps Canvas App (UI Layer)

Auto-generated screens: Employee Form, Leave Request Gallery, Approval Dashboard

Power Automate (Workflow Layer)

AI-generated approval flows, notifications, escalation rules

Microsoft Dataverse (Data Layer)

Tables: Employee, LeaveRequest, LeaveType | Relationships | Business Rules | Security

Azure Cloud Infrastructure

Global scale, compliance, encryption, 99.9% SLA



Copilot doesn't replace the developer — it accelerates the developer. Your Dataverse knowledge makes the difference.

Live Demo: Copilot Prompt → Dataverse Schema → App

User Prompt

"Create an HR application that lets me:

- 1. Manage employee records*
- 2. Track department assignments*
- 3. Handle leave requests with approval*
- 4. View employee project assignments*

Include manager hierarchy."



Copilot Generated Schema

Tables Created:

- Employee (15 columns)
- Department (8 columns)
- LeaveRequest (12 columns)
- Project (10 columns)

Relationships:

- Employee ↔ Department (N:1)
- Employee → Manager (Self-ref)
- Employee ↔ Project (N:N)

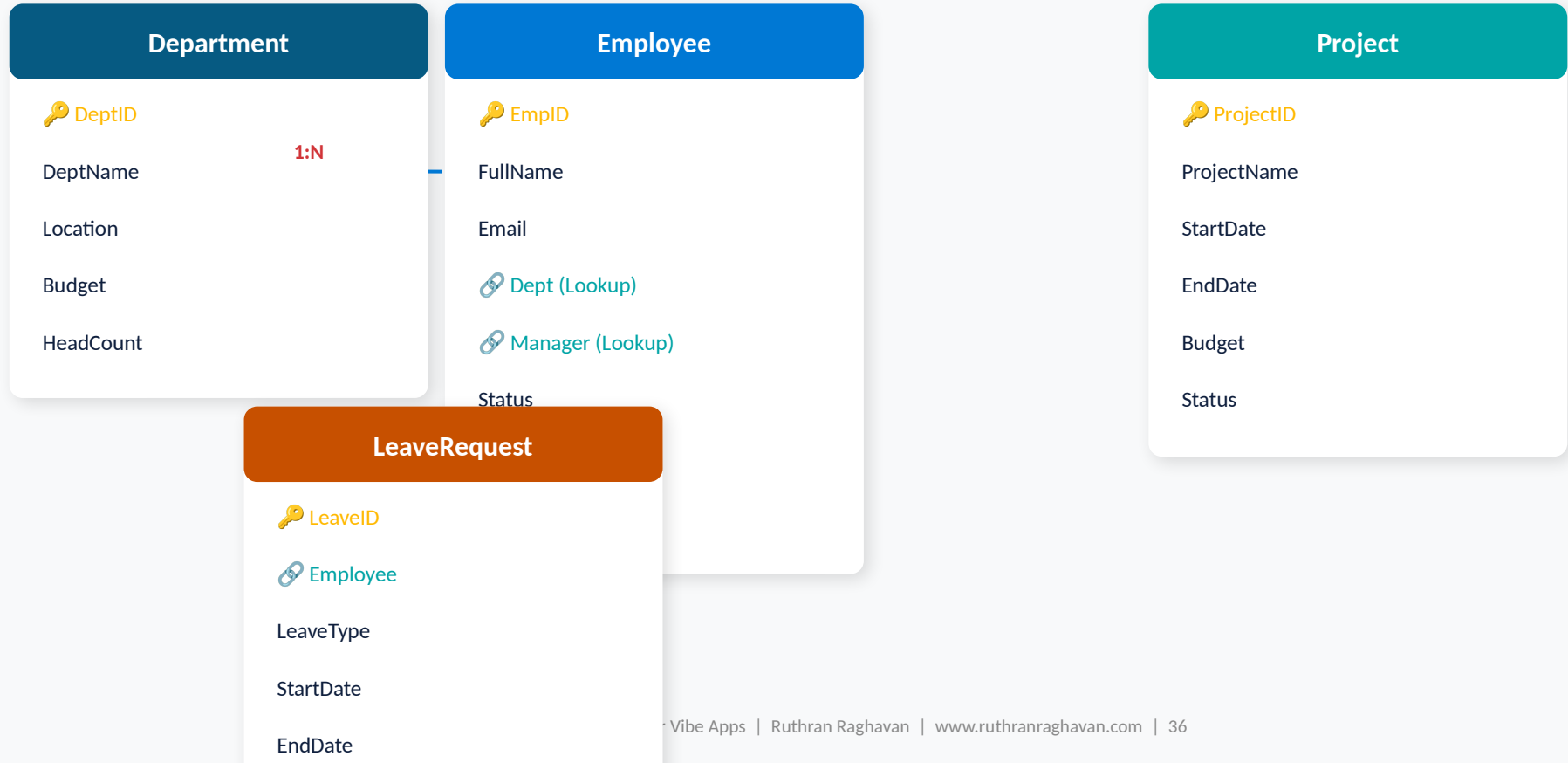
 App Screens: 6 generated

07

End-to-End HR System

Building a complete HR Management System with Dataverse

HR System Design — Data Model



Build the HR System — Step by Step

01

Create a Solution

make.powerapps.com → Solutions → New Solution. Set publisher prefix to 'hr'.

03

Create Employee Table

New Table → 'Employee'. Add columns: FullName, Email, Status (Choice), StartDate (Date), Salary (Currency).

05

Add Manager Lookup

In Employee table, add another Lookup column → also points to Employee table (self-referential manager).

07

Create Leave Request Table

Add LeaveRequest table with Lookup to Employee, Choice for LeaveType, Date fields, Status Choice.

02

Create Department Table

New Table → 'Department'. Add columns: DeptName (Text), Location (Text), Budget (Currency).

04

Add Department Lookup

In Employee table, add Lookup column → points to Department table. This creates the 1:N relationship.

06

Create Project + N:N

Create Project table. Add N:N relationship: Employee ↔ Project via table relationship designer.

08

Generate App with Copilot

From Solution, add Canvas App. Use Copilot to describe the HR app. Review and accept generated screens.

08

Knowledge Check & Assessment

Testing your Dataverse fundamentals

Knowledge Check — Questions 1-5

1. What is the current Microsoft terminology for 'Entities' in Dataverse?

- A. Collections
- B. Tables ✓**
- C. Objects
- D. Containers

2. Which relationship type is created by a Lookup column?

- A. N:N
- B. 1:1
- C. 1:N (One-to-Many) ✓**
- D. Hierarchical

3. Which Power Fx function is used to CREATE a new Dataverse record?

- A. Insert()
- B. Create()
- C. NewRecord()
- D. Patch() with Defaults() ✓**

4. What is a Global Choice in Dataverse?

- A. A choice column shared across multiple tables ✓**
- B. A choice with more than 10 options
- C. A lookup to a global table
- D. A system-defined choice

5. Which environment strategy is recommended for schema changes?

- A. Make changes in Production directly
- B. Development → Test → Production ✓**
- C. Test → Production → Development
- D. Only use Development environment

Knowledge Check — Questions 6–10

6. What does CRUD stand for in data operations?

A. Create, Run, Update, Deploy B. Create, Read, Update, Delete ✓ C. Copy, Read, Upload, Delete D. Create, Retrieve, Undo, Delete

7. Which column type should you use for 'Employment Status' with fixed options?

A. Text B. Lookup C. Choice ✓ D. Boolean

8. In a 1:N relationship between Department and Employee, where is the Lookup column?

A. On the Department table B. On the Employee table ✓ C. On both tables D. On a separate junction table

9. Which layer in Dataverse security controls what tables a user can access?

A. Field-Level Security B. Environment Security C. Security Roles ✓ D. Business Units

10. What happens when you add a new column to a Dataverse table?

A. All apps automatically get the column added ✓ B. Existing apps break immediately C. Only Model-Driven Apps update D. You must recreate the table

Scenario-Based Questions

Scenario 1: Retail Company

A retail company has Products, Categories, Suppliers, and Orders. One supplier provides many products. One order contains many products.

Q: What relationships would you design? What column types would you use for 'Product Price' and 'Order Status'?

Scenario 2: Training Department

HR wants to track which training courses each employee has completed. Employees can complete many courses. Each course can be completed by many employees.

Q: What relationship type is this? How would you store the completion date for each employee-course pair?

Scenario 3: Schema Change

After 6 months in production, HR wants to add a 'Performance Rating' (1-5 scale) column to the Employee table.

Q: What column type would you use? What is the risk of adding this column? How do you handle the ALM process?

Hands-On Exercises

Exercise 01

Build the Department Table

 10 min

1. Create a Solution in your environment
2. Add a custom table: Department
3. Add columns: DeptName (Text), Location (Text), HeadCount (Whole Number), Budget (Currency)
4. Add 3 sample department records

Exercise 02

Build the Employee Table with Lookup

 15 min

1. Create Employee table with: FullName, Email, Status (Choice: Active/Inactive), StartDate (Date)
2. Add a Lookup column to Department table
3. Add 5 sample employee records linked to departments

Exercise 03

Create a Canvas App from Dataverse

 15 min

1. New Canvas App → From Dataverse → Employee table
2. Observe auto-generated screens
3. Add a Search bar above the gallery
4. Connect search to Filter(Employees, ...)

09

Capstone Lab

Build a complete HR Management System from scratch

Capstone Lab — HR Management System

Build a fully functional HR Management System using Dataverse and Power Apps. This lab encompasses everything covered in this module.

Tables Required

- ✓ Department Table (5+ columns)
- ✓ Employee Table (8+ columns, Status Choice)
- ✓ Project Table (6+ columns)
- ✓ Leave Request Table (7+ columns)

Relationships Required

- ✓ Department → Employee (1:N via Lookup)
- ✓ Employee → Manager (Self-referential)
- ✓ Employee ↔ Project (N:N)
- ✓ Employee → Leave Request (1:N)

App Requirements

- ✓ Employee List Gallery with Search
- ✓ Add/Edit Employee Form (Patch)
- ✓ Department filter dropdown
- ✓ Leave Request submission form

Bonus Challenges

- ★ Business rule: auto-set End Date when Terminated
- ★ Security role with read-only for staff
- ★ Copilot-generated screen
- ★ Power Automate leave approval flow

Capstone Assessment Rubric

Criterion	Excellent (4)	Proficient (3)	Developing (2)	Beginning (1)
Table Design	4+ tables, all required columns, correct types	3 tables, most columns correct	2 tables, some columns	1 table only
Relationships	All 4 relationships configured correctly	3 relationships correct	2 relationships	1 relationship or none
CRUD Operations	Create, Read, Update, Delete all working	Create and Read working	Create only working	No operations working
Lookups/Choices	All lookup & choice columns configured	Most configured	Some configured	None configured
App Functionality	All screens, search & filter working	Most screens working	1-2 screens working	App generated but broken
Naming Conventions	Publisher prefix on all custom elements	Most elements prefixed	Some prefixed	No naming conventions

10

Summary & Key Takeaways

What you've learned and where to go next

Key Takeaways



Dataverse is more than a database

01



It's a complete data platform with security, business logic, APIs, and AI built in.



Tables, Columns, Rows — not Entities & Fields

02



Microsoft's current terminology. Rows are records. Choose data types thoughtfully.



Relationships are the heart of good data design

03



1:N via Lookup is the most common. N:N creates a junction table automatically.



Patch() is your Create & Update Swiss Army Knife

04

Defaults() = insert. Record reference = update. Filter() + Search() for reading.

Security is layered and powerful

05

Environment → Roles → Field-Level → Record Sharing. Use Security Roles always.



Vibe Apps + Dataverse = The Future of Low-Code

06



AI generates 70% of the app. Your Dataverse expertise makes the remaining 30% possible.

Dataverse Quick Reference Cheat Sheet



TERMINOLOGY

New Term → Old Term

Table → Entity

Column → Field / Attribute

Row → Record

Choice → Option Set

Lookup → FK Column

Environment → Instance



POWER FX

CREATE

Patch(T, Defaults(T), {...})

UPDATE

Patch(T, record, {...})

DELETE

Remove(T, record)

READ ALL

Filter(T, condition)

READ ONE

LookUp(T, ID=val)

SEARCH

Search(T, txt, 'col')



RELATIONSHIPS

1:N

Lookup on MANY side

Dept → Employees

N:1

Lookup on child table

Employee → Dept

N:N

Auto junction table

Emp ↔ Project

Self-ref

Lookup → same table

Manager → Employee

Resources & Further Learning

Official Microsoft Docs

- learn.microsoft.com/power-apps/maker/data-platform/
- learn.microsoft.com/power-apps/maker/data-platform/data-platform-intro
- learn.microsoft.com/power-platform/admin/create-environment

Microsoft Learn Paths

- Create model-driven apps in Power Apps
- Work with Microsoft Dataverse
- PL-900: Microsoft Power Platform Fundamentals
- PL-400: Microsoft Power Platform Developer

Vibe Coding & Copilot

- learn.microsoft.com/power-apps/maker/canvas-apps/ai-overview
- Microsoft Copilot Studio documentation
- Power Platform Copilot YouTube channel

Instructor Resources

- Website: www.ruthranraghavan.com
- Training programs and advanced courses
- LinkedIn: Ruthran Raghavan — Chief AI Scientist



Thank You!

You have completed Module 6: Dataverse Fundamentals for Vibe Apps

RR

Ruthran Raghavan

Chief AI Scientist

www.ruthranraghavan.com